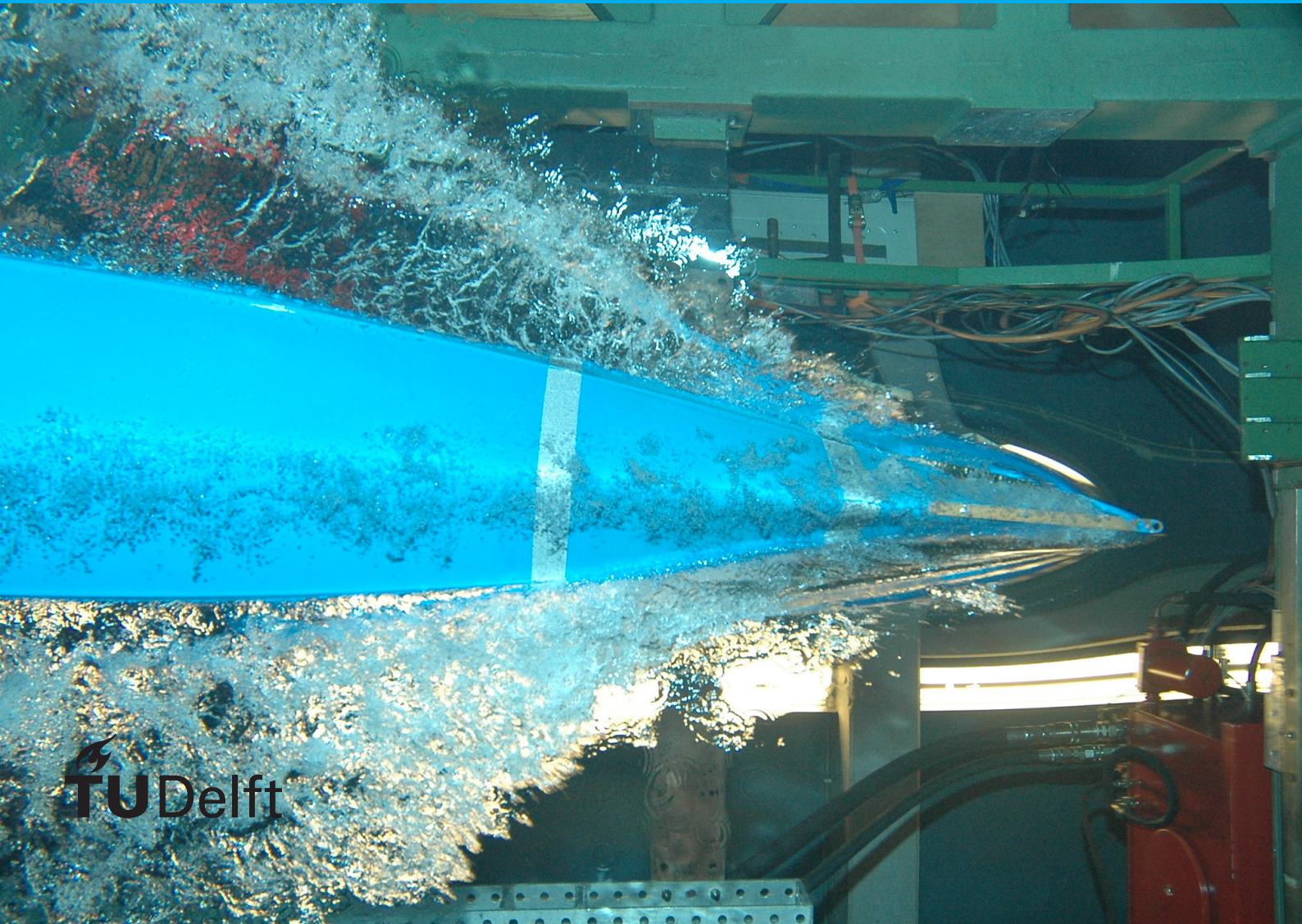
Orbital ascent trajectory optimization using reinforcement learning

Preliminary thesis

A. M. de Windt



Orbital ascent trajectory optimization using reinforcement learning

Preliminary thesis

by

A. M. de Windt

to obtain the degree of Master of Science
at the Delft University of Technology,
to be defended publicly on Tuesday January 1, 2013 at 10:00 AM.

Student number:	1234567	
Project duration:	March 1, 2012 – January 1, 2013	
Thesis committee:	Prof. dr. ir. J. Doe,	TU Delft, supervisor
	Dr. E. L. Brown,	TU Delft
	Ir. A. Aaronson,	Acme Corporation

This thesis is confidential and cannot be made public until December 31, 2013.

An electronic version of this thesis is available at `http://repository.tudelft.nl/`.

TU Delft

Todo list

Rewite this paragraph. 4
Rewrite this paragraph. 6
Add reference to the transition probability function explanation in previous chapter. 6
Fix the tuple 10
Update captions 11

Preface

Preface…

A. M. de Windt
Delft, January 2013

List of Figures

2.1 Example reward distribution for a 5-armed bandit problem. 6
2.2 Average reward of 2000 10-armed bandit problems for 1000 episodes when following the greedy and two ϵ-greedy policies. 7
2.3 Example reward distribution for a 5-armed bandit problem. 11
2.4 Example reward distribution for a 5-armed bandit problem. 11

List of Tables

Contents

List of Figures vii

List of Tables ix

1 Introduction 1

2 Introduction into Reinforcement learning 3
- 2.1 Finite Markov Decision Process . . . 3
- 2.2 Elements of Reinforcement Learning . . . 3
- 2.3 Expected return . . . 4
- 2.4 Policy and value functions . . . 5
- 2.5 K-armed bandit problem . . . 5
- 2.6 ϵ–greedy policies . . . 5
- 2.7 Dynamic Programming . . . 6
  - 2.7.1 Policy evaluation . . . 7
  - 2.7.2 Policy improvement . . . 8
  - 2.7.3 Policy iteration . . . 9
  - 2.7.4 Generalized Policy Iteration . . . 9
- 2.8 Monte Carlo Methods . . . 9
- 2.9 Temporal Difference . . . 10
  - 2.9.1 TD Prediction . . . 10
  - 2.9.2 Sarsa: On-policy TD Control . . . 10
  - 2.9.3 Q-learning: Off-policy TD control . . . 10

3 Approximate solutions 13
- 3.1 Tile-Coding . . . 13

4 Past, present and future of Reinforcement Learning 15

5 Conclusion 17

6 Proposed thesis research 19

Bibliography 21

1 Introduction

Thanks to advancements in technology and an increasingly widespread use of satellite services, demand for space access has been steadily rising the past decades. This growth in the satellite launch services market has lead to the development of new launchers by both government agencies and an emerging private sector.

One often overlooked, but crucial step during the design process of any product is the performance evaluation of proposed designs. This is the process of determining a products ability to meet the design requirements. Too low of a performance, and the product will not be able to fullfil its purpose. To high of a performance and the product may be over engineered or inefficient.

In the case of a launcher the performance may be measured by how much payload mass it can bring to it's target range of orbits. In other words, in order to maximize the performance of a launcher, the propellant mass fraction of the upper stage needs to be reduced.

A successful orbital insertion requires the launcher to carefully follow an optimized trajectory. This trajectory has a massive effect on the performance of the launcher and is unfortunately not a trivial problem to solve. There has been quite some research in this trajectory optimization problem, but due to the complexity, the most reliable way to get the optimum is by using a global optimization method such as Genetic Algorithm (GA) or Participle Swarm Optimization (PSO).

Using a global optimizer to solve this problem is computationally intensive. This is not an issue when computing the trajectory for one particular launch during normal operations, but it is during the design the launch vehicle. A fast trajectory optimizer is desired during the design process in order to for example get a large number of results when performing a sensitivity analysis on the design parameters or continuously evaluate and monitor the feasibility of a changing design.

This thesis proposes the use of Reinforcement Learning (RL) as a possible solution to the trajectory optimization problem. With enough training an RL agent is able to learn to solve complex and non-linear problems. The goal is to sufficiently train an RL agent in order for it to be able to compute the optimal or close to optimal trajectory of a particular launcher design to a specific target orbit with just one simulation run.

Even if generating optimal trajectories using RL ends up being unfeasible, it may still generate close to optimal trajectories, which may either be acceptable for a design parameter sensitivity analysis, or serve as an initial guess for a more refined optimization algorithm.

The use of Reinforcement Learning (RL) in spacecraft guidance, navigation and control opens up the possibilities to solve future problems as space exploration becomes more prevalent. For example thanks to it's ability to run and adapt in real time, it may be used in situations that are difficult to forsee or too cumbersome to manually take into account, such as hovering and manoeuvering around the non-uniform, rotating gravitational field of an asteroid.

The objective of this thesis is to develop a orbital ascent trajectory optimization method using reinforcement learning and investigate its effectiveness compared to existing methods.

What is Reinforcement Learning and what can it be used for?

- How does RL work?
- What is RL capable of?
- How has RL been used so far?
- What state of the art technological developments make use of RL?

How are orbital ascent trajectories currently optimized?

- What are the challenges faced when optimizing an orbital ascent trajectory?
- What methods are currently used or being researched to solve this problem?

How can Reinforcement Learning methods be used to solve the orbital ascent trajectory optimization problem?

- What advantages do RL methods have over existing methods?
- What RL methods have the potential of being effective in solving this problem?
- How do these newly developed methods compare to existing methods?
- May these RL methods be used to compliment or improve the performance of existing methods?

2

Introduction into Reinforcement learning

Reinforcement Learning (RL) is a method of machine learning different from other machine learning methods such as supervised learning which uses a set of labeled examples from a knowledgeable supervisor, and unsupervised learning which is typically used to find hidden patterns in unlabeled data. In RL the learner is not told what action to take to achieve its goal, but instead it maps situations and actions by trying them out and discovering the most valuable. It does this by not only looking at the reward it received by its latest action, but it also takes into account any future rewards it could receive in the future by taking that action.

2.1. Finite Markov Decision Process

A Markov Decision Process (MDP) is a stochastic process that provides a mathematical framework used to describe problems where a learner learns from its actions to achieve an end goal. This framework is used in many areas, such as robotics, economics, manufacturing and in the case of this report, Reinforcement Learning.

The two main components of a Markov Decision Process (MDP) are the agent, which makes decisions, interacts with its environment and learns. And the environment which consists of everything else. The two are constantly interacting with each other, the agent will perform an action which changes the state of the environment and the environment responds by giving it a reward. The goal of the agent is to maximize the reward it receives from the environment.

This happens in a sequence of discrete time steps. At the beginning of each time step the environment is at a certain state s. The agent then decides based on its policy π which action a to take, this in turn changes the environment to the new state s' and the environment then gives the agent a certain reward r. In order for it to be a MDP this transition and the reward must satisfy the Marcov property, which means it needs to be memoryless and thus the environment's behavior must be independent of its past states. In other words, the probability of the environment to go from state s to s' and the reward r it gives back to the agent, must only be dependent on the initial state s and action a.

2.2. Elements of Reinforcement Learning

The main goal when solving an MDP problem is to find a policy π that defines the action $\pi(s)$ the agent will take in order to maximize the sum of rewards it receives. It characterizes the way in which the agent will behave at a given time by mapping the agents perceived state of the environment to the actions taken in those states. This policy may be as simple as a table or consist of several Neural Networks. The policy is the core of the agent in that it alone defines the behavior of the agent.

On each time step the environment will send the agent a reward. The goal of the agent is to maximize this reward in the long run. Thus this reward is used to tell the agent which states are good or bad. If the agent receives a low reward for a certain action it can change the policy so it takes a different action in the future.

The reward signal tells the agent what the immediate effects of its action are. However, its goal is to maximize the cumulative sum of rewards it receives over time, and thus in order for it to receive higher rewards in the future, it might be necessary for the agent to take actions that yield lower rewards with the promise that it will receive higher rewards later. So the value is an indication of the reward the agent can expect to receive in the future by taking a certain action, and thus it is the main parameter used in the decision making process.

By defining the state and action taken during step t as S_t and A_t respectively, and its corresponding reward as R_t we can define the true value of an arbitrary action a as:

$$q_*(s,a) \doteq \mathbb{E}\left[R_t \mid S_t = s, A_t = a\right] \tag{2.1}$$

If $q_*(s,a)$ was known, then the solution would be trivial. Just select the action with the highest value. So it's assumed that the action values are not known, but are instead estimated from the rewards the agent receives. The estimated value of the state-action pair (s,a) at time t is known as $Q_t(s,a)$, thus $Q_t(s,a)$ needs to be as close to $q_*(s,a)$ as possible.

Finally some RL agents make use of a model for planning. This allows these agents to choose actions based on possible future situations before actually being experienced.

2.3. Expected return

The goal of the agent is to maximize the reward it receives over time. In order to formalize this goal, the *expected return* G_t can be defined as a function of the sequence of rewards. In it's simplest form this can be the cumulative sum of all rewards after time step t until the final time step T.

$$G_t \doteq R_{t+1} + R_{t+2} + R_{t+3} + \ldots + R_T \tag{2.2}$$

This function assumes that there are one or more terminal states where the agent-environment interaction ends and break naturally into subsequences called *episodes*. A good example are plays of a game or any type of repeating task. Even if there are different terminal states, eg. winning or losing a game, the starting state of the next episode must be independent of the previous episode. These type of tasks are called *episodic tasks*.

However not all tasks are episodic, it is possible an RL-agent needs to solve some task that can continuo without a limit. These types of tasks are called *continuing tasks*. The expected return given by Eq. 2.2 can be problematic for these types of tasks, since the final time step would be at $T = \infty$. Thus it is possible the value it's trying to maximize is infinite as well. Thus to solve this problem the rewards can be $discounted$. In this approach the sum of discounted rewards, also known as the *discounted expected return*, shown in Eq. 2.3 is maximized.

his paragraph.

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \tag{2.3}$$

It is possible to unify the expected return equations for the episodic and continuing tasks into a single one. For example, in the transition diagram for a continuos task shown below the environment transitions from states s_0 to s_1 and after its next action it ends up at a special absorbing state where all actions will result in the same state without any reward. This results in reward sequence $+1, +1, +1, 0, 0, \ldots$. By summing the whole infinite sequence we get the same result as summing the first T rewards, where $T = 3$ in this case. This is also true with discounting.

Thus Eq. 2.4 shows the unified equation for the (discounted) expected return for both episodic and continuing tasks. If the task at hand is continuing, then $T = \infty$, while for episodic tasks T will be some finite integer. It is possible to have $T = \infty$ for episodic tasks, but due to the reasons mentioned before the expected return must be discounted for continuing tasks, so $\gamma = 1$ and $T = \infty$ cannot true at the same time.

$$G_t \doteq \sum_{k=t+1}^{T} \gamma^{k-t-1} R_k \tag{2.4}$$

One important property of the expected return used in reinforcement learning is that it is recursive as demonstrated below.

$$\begin{aligned} G_t &= R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \dots \\ &= R_{t+1} + \gamma \left(R_{t+2} + \gamma R_{t+3} + \gamma^2 R_{t+4} + \dots \right) \\ &= R_{t+1} + \gamma G_{t+1} \end{aligned} \tag{2.5}$$

2.4. Policy and value functions

The policy is a mapping from the state of the environment to the probabilities of the agent selecting each action. If the policy at time t is π, then $\pi(a|s)$ is the probability of selecting action a if the current state is s.

The value of a state can be different depending on the policy of the RL-agent, since different policies will result in different actions and thus different states and rewards. So $v_\pi(s)$ is the expected return when starting at state s and following policy π. v_π can be defined by:

$$v_\pi(s) = \mathbb{E}_\pi \left[G_t | S_t = s \right] = \mathbb{E}_\pi \left[\sum_{k=t+1}^{T} \gamma^{k-t-1} R_k \middle| S_t = s \right], \text{ for all } s \text{ in } \mathcal{S} \tag{2.6}$$

For some problems it might be more suitable to look at the value of state-action pairs. Thus similar as to the state value above, the state-action value under policy π, denoted by $q_\pi(s,a)$, is defined by:

$$q_\pi(s,a) = \mathbb{E}_\pi \left[G_t | S_t = s, A_t = a \right] = \mathbb{E}_\pi \left[\sum_{k=t+1}^{T} \gamma^{k-t-1} R_k \middle| S_t = s, A_t = a \right], \text{ for all } s \text{ in } \mathcal{S} \tag{2.7}$$

2.5. K-armed bandit problem

K-armed bandit problems will be used as examples in the following sections. These are simple learning problems where the agent is given the option to take k different action's. There is only one state and the episodes terminate after the agent has taken it's action. Furthermore the examples in this report use zero mean, unit variance normal distributions to select the true value $q_*(a)$ of the actions. The rewards given by the environment are then selected with $q_*(a)$ mean and unit variance normal distribution. Figure 2.1 shows an example reward distribution for a 5-armed bandit.

2.6. ϵ–greedy policies

The main difference between Reinforcement Learning and other types of machine learning is that RL uses training information that evaluates the actions taken by the agent instead of a set of correct actions. This means that in order to determine $Q_t(a)$ the agent needs to explore and try out as many actions as possible in order to find good behavior. To do this the agent can take random actions and learn from the rewards it gets back. This is called *exploration*.

However, the main goal of the agent is to maximize the rewards it receives from the environment over time. To achieve this goal the agent can choose to take the greedy actions. These are the actions with the highest value in $Q_t(a)$. This process of choosing the greedy action is known as *exploitation* since the agent is exploiting its knowledge to achieve its goal.

Unfortunately exploration and exploitation conflict with each other since in the former the agent wants to try out all possible actions, but only wants to perform a specific action if it's following a greedy policy. Thus a possible compromise is to use what is known as an ϵ-greedy policy. In these policies, the agent

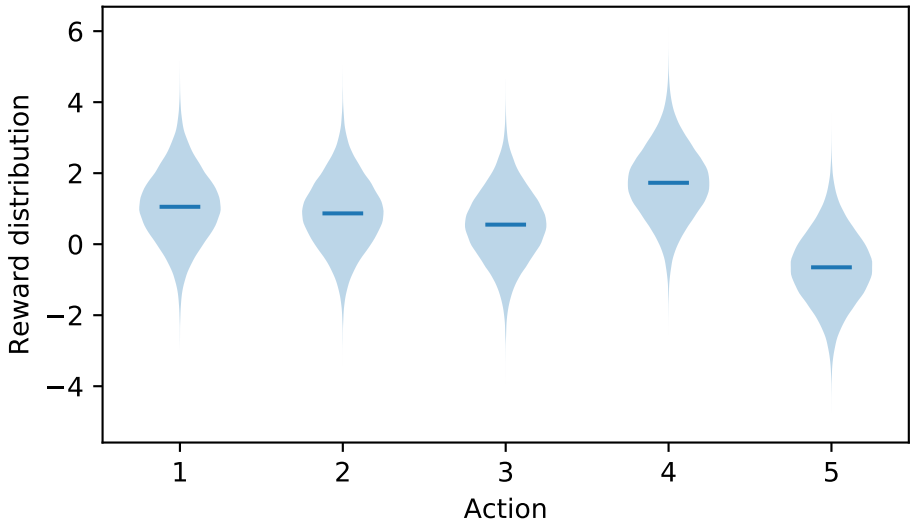


Figure 2.1: Example reward distribution for a 5-armed bandit problem.

will choose a random action ϵ percentage of the time and the greedy action the rest of the time. This allows the agent to explore and try out new actions or old actions whose value might have changed, while also trying to achieve it's goal.

this paragraph.

Since the agent is trying out new actions while following ϵ-greedy policies, it is able to learn faster and more than if it was following a greedy policy. To demonstrate this 2000 10-armed bandit problems are run for 1000 episodes. The average reward of all bandits are stored after each episode. Figure 2.2 shows the average reward and the percentage of agents that selected the optimal action for the greedy policy and two ϵ-greedy policies ($\epsilon = 0.1$ and $\epsilon = 0.1$). A full explanation of the learning method used can be found in the section **??**, but in short, the action value function $Q_t(a)$ is estimated by averaging the rewards received from the environment.

As it can be seen when using ϵ-greedy policies the agents learn faster and are thus able to receive higher rewards sooner. On the other hand, while following the greedy policy, the agents will quickly choose the first action they find with a reward higher than the initial action value and then perform that action on all episodes. As a results many of the agents are not able to find the optimal policy.

2.7. Dynamic Programming

Dynamic Programming (DP) refers to a collection of methods used to solve MDP problems. The main idea of DP and RL is the use of value functions to organize and structure the search for good policies. The optimal policies can be easily be found once the optimal value functions v_* or q_* which satisfy the Bellman optimality equations shown below are known.

rence to the
n probability func-
anation in previ-
pter.

$$\begin{aligned} v_*(s) &= \max_a \mathbb{E}\left[R_{t+1} + \gamma v_*(S_{t+1}) | S_t = s, A_t = a\right] \\ &= \sum_{s',r} p(s', r|s, a)\left[r + \gamma v_*(s')\right] \end{aligned} \tag{2.8}$$

$$\begin{aligned} q_*(s,a) &= \mathbb{E}\left[R_{t+1} + \gamma \max_{a'} q_*(S_{t+1}, a') | S_t = s, A_t = a\right] \\ &= \sum_{s',r} p(s', r|s, a)\left[r + \gamma \max_{a'} q_*(S_{t+1}, a')\right] \end{aligned} \tag{2.9}$$

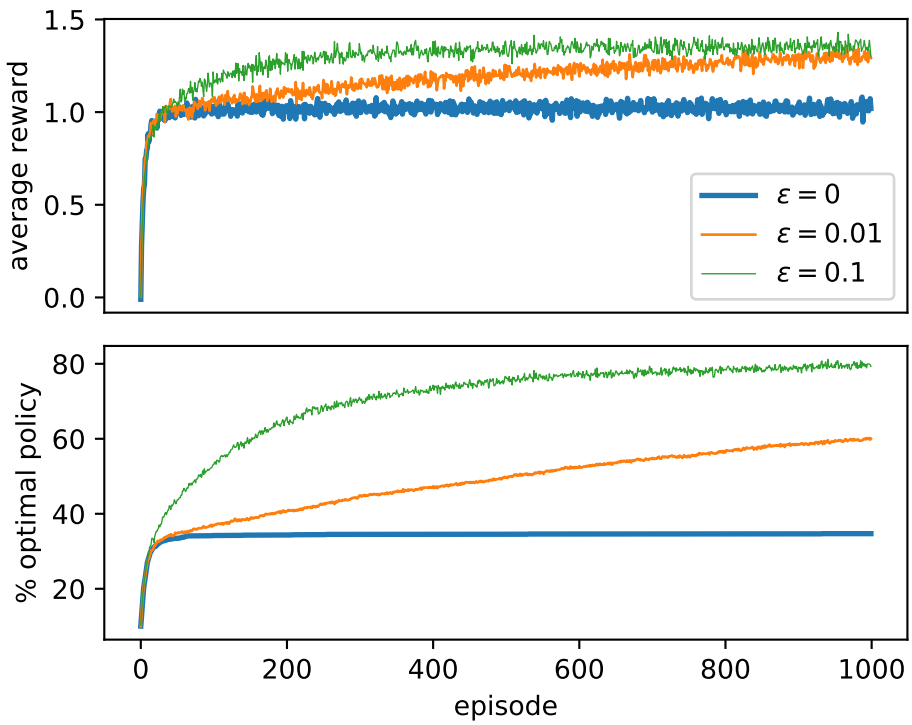


Figure 2.2: Average reward of 2000 10-armed bandit problems for 1000 episodes when following the greedy and two ϵ-greedy policies.

DP algorithms are found by turning the Bellman equations into assignment and, or update rules for improving approximations of the value functions.

2.7.1. Policy evaluation

The first step in DP is to compute the *state-value* function v_π for an arbitrary policy π. This is called *policy evaluation* or *prediction problem*.

This can be done by writing the state-value function v_π from eq. 2.6 in terms of the policy probability function $\pi(a|s)$ and the transition probability function $p(s', t|s, a)$. The policy probability function is the probability of action a being selected by the policy while being in state s. And the transition probability function gives the probability of the environment transitioning to state s' and giving reward r if action a is taken while in state s.

$$\begin{aligned} v_\pi(s) &\doteq \mathbb{E}_\pi\left[G_t|S_t = s\right] \\ &= \mathbb{E}_\pi\left[R_{t+1} + \gamma G_{t+1}|S_t = s\right] \\ &= \mathbb{E}_\pi\left[R_{t+1} + \gamma v_\pi(S_{t+1})|S_t = s\right] \\ &= \sum_a \pi(a|s) \sum_{s',r} p(s', t|s, a)\left[r + \gamma v_\pi(s')\right] \end{aligned} \tag{2.10}$$

If the environment's dynamics are known, then Eq. 2.10 is a system of $|\mathcal{S}|$ equations with $|\mathcal{S}|$ unknowns. Solving it would be tedious, but trivial. However, this is almost never the case, thus iterative solution methods provide a better approach. Consider a sequence of value functions $v_1, v_2, v_3, \ldots$, each mapping $\mathcal{S}^+$ to $\mathbb{R}$. The initial value function approximation may be chosen arbitrarily and new approximations are calculated using the Bellman equation for v_π as an update rule.

$$\begin{aligned} v_{k+1}(s) &= \mathbb{E}\left[R_{t+1} + \gamma v_k(S_{t+1}) | S_t = s, A_t = a\right] \\ &= \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a)\left[r + \gamma v_k(s')\right] \end{aligned} \tag{2.11}$$

The state-value function v_π is a fixed point for this update rule because the Bellman equation for v_π assures us of equality in this case. It is also possible to show that the sequence v_k approaches v_π as $k \to \infty$. This algorithm is called the *iterative policy evaluation.*

To produce each approximation the iterative policy evaluation applies the operation to each state s and replacing the old value of s with a new one based on the immediate received reward and the old values of the successor states of state s. This operation is called an *expected update*.

One way to implement this algorithm is to keep two arrays, one to hold the old values and another for the new ones. This way the new values can be computed from the old ones, without changing the old ones. It is also possible to implement it with one array and update the values "in place", by immediately replacing the old value by the one. The in-place algorithm not only consumes less memory, but also usually converges faster than the two-array version. This is because the new values are used as soon as they are available.

2.7.2. Policy improvement

With the value function $v_\pi(s)$ for a deterministic policy π known, it is possible to check for each state, whether there is an action a which yields a larger value than the action taken by the current policy π. One way to do this, is for each state s, select an action a different from the action taken by the current policy ($a \neq \pi(s)$) and calculate the value if that action is taken first, followed by the current policy after that. The value for this behavior is:

$$\begin{aligned} q_\pi(s,a) &= \mathbb{E}\left[R_{t+1} + \gamma v_\pi\left(S_{t+1}\right) | S_t = s, A_t = a\right] \\ &= \sum_{s',r} p(s',r|s,a)\left[r + \gamma v_\pi\left(s'\right)\right] \end{aligned} \tag{2.12}$$

If this value is greater than that of the current policy, then a better action has been found. It can therefore be expected that a policy where the new action a is taken whenever state s is encountered is a better policy than the current policy. Thus updating the policy π with the new action will result in a new policy that is as good as or better than the current policy.

Repeating this process for all state-action combinations, it is possible to find a new greedy policy π', given below. Where $\arg\max_a$ denotes the value of a which maximizes the expression that follows.

$$\begin{aligned} q_\pi(s,a) &\doteq \arg\max_a \; q_\pi(s,a) \\ &= \arg\max_a \; \mathbb{E}\left[R_{t+1} + \gamma v_\pi\left(S_{t+1}\right) | S_t = s, A_t = a\right] \\ &= \arg\max_a \; \sum_{s',r} p(s',r|s,a)\left[r + \gamma v_\pi\left(s'\right)\right] \end{aligned} \tag{2.13}$$

The process of making a new policy that improves over the original one using the values of the original policy is called *policy improvement*. In the case where the new policy π' is as good than the old policy π, Then $v_{\pi'} = v_\pi$ and from there is follows that for all $s \in \mathcal{S}$:

$$\begin{aligned} v_{\pi'}(s) &= \max_a \; \mathbb{E}\left[R_{t+1} + \gamma v_{\pi'}(S_{t+1}) | S_t = s, A_t = a\right] \\ &= \sum_{s',r} p(s',r|s,a)\left[r + \gamma v_{\pi'}(s')\right] \end{aligned} \tag{2.14}$$

This is the same as the Bellman optimality equation (2.8) and thus $v_{\pi'}(s)$ is the optimal value function $v_*(s)$. So policy improvement will always give a better policy, except when the policy is already optimal.

Policy improvement can also be applied to stochastic policies. These are policies where each action is given a probability $\pi(a|s)$ to occur. If there are ties between actions, that is, multiple actions that result in the same maximum value, then each of these action can be given a part of the probability to occur, as long as any non-optimal action is given zero probability.

2.7.3. Policy iteration

Using policy improvement a policy, π, can be improved using v_π to yield a better policy, π'. With this new policy a new *state-value* function v'_π can be evaluated from π', which results a new improved policy π''. This process can be repeated indefinitely.

Each new policy in guaranteed to be in improvement over the previous one unless it's already the optimal policy. Since a finite MDP only had a finite number of policies, this process must converge to the optimal policy and the optimal value function in a finite number of iterations.

This way of finding the optimal policy it called policy iteration.

2.7.4. Generalized Policy Iteration

In Policy Iteration the Policy Improvement and Policy Evaluation processes alternate, each completing before the other one. However this is not necessary, as long both processes continue to update all states, the ultimate result is almost always the same, convergence to the optimal value function and optimal policy. The term Generalized Policy Iteration (GPI) is used to refer the general idea of letting the Policy Improvement and Policy Evaluation processes interact independent of the granularity and other details of the other processes.

2.8. Monte Carlo Methods

It is not always possible to have a model fully describing the system for which the optimal policy is being discovered. This is where Monte Carlo methods come into play. Unlike Dynamic Programming, Monte Carlo methods require only *experience*, sample sequences of states, actions and rewards or simulated interaction with the environment. Thus no prior knowledge of the dynamics are needed to determine the optimal policy. These methods allow learning from simulated experience, which would require a model, but this model only needs to generate sample transitions and not the complete probability distribution of all possible transitions.

Just like the k-armed bandit problem demonstrated in Section **??**, Monte Carlo methods work by averaging the returns for state—action pairs. The main difference is that there are multiple states, each acting like a different, interrelated bandit problems, that is, the return from taking an action in one state depends on the actions taken in future states in the same episode.

Since the agent is learning and changing its policy throughout the process, the action selection is constantly changing and thus the problem is nonstationary from the point of view of the earlier state. To handle the nonstationarity, the idea of Generalized Policy Iteration (GPI) is adapted. In DP the value functions are computed directly from the knowledge of the MDP. With Monte Carlo methods however the value functions are *learned* from sample returns with the MDP. The value functions and their policies still interact in the same way.

As previously mentioned, the value of a state ($v_\pi(s)$) is the expected return, the expected cumulative future discounted reward, starting from that state. The obvious way to estimate it from experience is to average all returns after visiting that state. As more returns are observed, the average will converge to the expected value. This is the underlying idea of all Monte Carlo methods.

If no model is available, then it might be better to estimate *action* values (the values of state—action pairs, $q_\pi(s,a)$). With a model, state values alone are sufficient to determine a policy, since the agent would simply have to look ahead one step and see which action results in the highest value. This is not possible without a model. The values of each action must be estimated, since the agent doesn't know to witch state each action will lead to.

2.9. Temporal Difference

Temporal-Difference (TD) learning is one of the main elements of reinforcement learning. It is a combination of ideas from Monte Carlo methods and DP. Just as with Monte Carlo methods, Temporal-Difference (TD) methods are able to learn from experience without the need of a model of the environment's dynamics, and just as with DP, TD methods update estimates based in part on other learned estimates, without waiting for a final outcome.

2.9.1. TD Prediction

Both TD and Monte Carlo methods use experience to solve the prediction problem. Given some experience following policy π, both methods update their estimate V of v_π for non-terminal states that occurred during that experience. Monte Carlo methods wait until the return G_t after visiting state S_t is known. A simple every-visit Monte-Carlo method is shown in Equation 2.15.

$$V(S_t) \leftarrow V(S_t) + \alpha\,[G_t - V(S_t)] \tag{2.15}$$

Monte-Carlo methods need to wait until the end of an episode since G_t is only known then. TD methods only need to wait until the next iteration by estimating G_t using the observed reward R_{t+1} and the estimated value $V(S_{t+1})$. Thus just like with DP, TD methods updates are based in part on existing estimates. These types of methods are known as bootstrapping methods.

Equation 2.16 shows the simplest TD method. This method is called TD(0) or one-step TD, because it's a special case of the TD(□) and n-step TD methods.

$$V(S_t) \leftarrow V(S_t) + \alpha\,[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)] \tag{2.16}$$

The quantity within the bracket is sort of an error, it's the difference between the estimated value of S_t and the better estimate $R_{t+1} + \gamma V(S_{t+1})$. This quantity is known as the TD error and can be found in various forms throughout reinforcement learning.

2.9.2. Sarsa: On-policy TD Control

In order to use TD prediction for the control problem the pattern of Generalized Policy Iteration (GPI) can be used, only this time using TD methods for the evaluation or prediction part. Approaches fall under two classes, on-policy and off-policy. This section focuses on an on-policy TD control method.

The first step is to learn an action-value function rather than a state-value function. For an on-policy method the estimated $q_\pi(s,a)$ is made for the behavior of the current policy. This can be done essentially the same as the TD method described above.

uple

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\,[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)] \tag{2.17}$$

This update is done after every iteration from a non-terminal state S_t. If S_{t+1} is a terminal state, then $Q(S_{t+1}, A_{t+1})$ is defined as zero. This rule uses every element of the quintuple of events, $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$, that make up a transition from one state-action to another. This quintuple gives rise to the name Sarsa for the algorithm.

2.9.3. Q-learning: Off-policy TD control

Another option is Q-learning, an off-policy TD control method. In this case the learned action-value function, directly approximates q_*, the optimal action-value function.

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)\right] \tag{2.18}$$

The advantage of this method is that it drastically simplifies the analysis of the algorithm and enables early convergence proofs. The only thing that is required is that all pairs continue to be updated. The

disadvantage is that since the action-value function learned is independent of the policy, the agent doesn't take into account the possibility of it taking a non-optimal action.

The following example is used to demonstrate the difference between on and off policy methods. Figure 2.3 shows a grid world. The agent's task to go from the start (S) block to the goal (G). However if the agent steps on the cliff, the episode will end, it receives a -100 reward and is then send back to the start. A -1 reward is given to agent each step until it has reached the goal.

Update captions

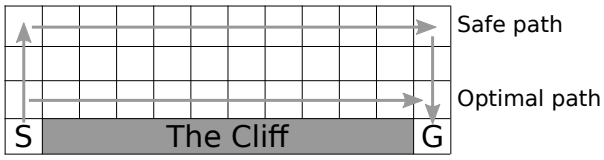


Figure 2.3: Example reward distribution for a 5-armed bandit problem.

200 agents are simulated for 500 episodes to demonstrate the difference between on and off policy learning methods. They all follow ϵ-greedy policies with $\epsilon = 0.1$, but half use Sarsa and the other half Q-learning. The Q-learning agents learn the optimal policy and thus travel right along the edge of the cliff. However since the agents may at any point take a step into any random direction, it is better to take the safer route further away from the edge. Since the Sarsa agents learn the roundabout policy, this is the path they learn to take. Figure 2.4 show the results of these simulations.

However once the agents have been trained and $\epsilon$is set to zero, the Q-learning agents outperform the Sarsa agents. With these new greedy policies the Q-learning agents will always receive a sum of rewards of -12 while the Sarsa agents receive -16 thanks to the longer path. If $\epsilon$is gradually reduced to zero, then both methods converge to the optimal policy.

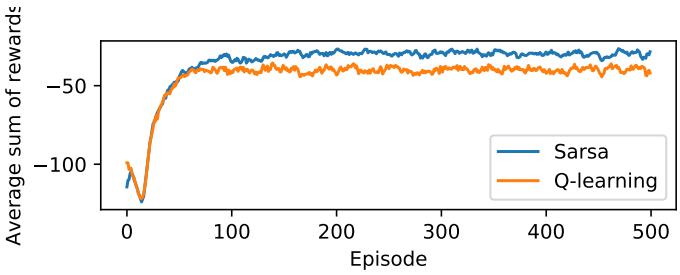


Figure 2.4: Example reward distribution for a 5-armed bandit problem.

3

Approximate solutions

Up until now all methods discussed used tables to store the learned value functions and policies. However these methods are not practical for many of the tasks where Reinforcement Learning (RL) is applied due these having large and continuos state spaces.

The problem with large state spaces is not just the memory needed to store the table, but also the time and amount of data needed to fill in these tables accurately. For many of these tasks the states encountered will never have been seen before, thus to make decisions it is necessary to generalize from previous encounters with different states that are similar to the current one. In other words the key issue is that of *generalization*. How can experience with a limited subset of states be generalized to produce a good approximation of a much larger subset?

Fortunately, generalization from examples has been extensively studied, and thus the problem to be solved is combining existing generalization methods with RL methods. The type of generalization methods required are often called *function approximation* because it take in samples of the target function and creates an approximation of the entire function.

The following chapters will cover several of these methods and will explain new issues which are introduced into RL when using function approximations.

3.1. Tile-Coding

4

Past, present and future of Reinforcement Learning

5

Conclusion

6

Proposed thesis research

Bibliography

[1] G. J. Alred, C. T. Brusaw, and W. E. Oliu. *Handbook of Technical Writing*. St. Martin's, New York, 2003 (seventh edition).

[2] Michel Goossens, Frank Mittelbach, and Sebastian Rahtz. *The LaTeX Companion*. Addison-Wesley, Reading, Mass., 1997.

[3] S. F. Gull. Developments in maximum-entropy data analysis. In J. Skilling, editor, *Maximum Entropy and Bayesian Methods*, pages 53–71. Kluwer Academic, Dordrecht, 1989.

[4] K. M. Hanson. Introduction to Bayesian image analysis. In M.H̃. Loew, editor, *Medical Imaging: Image Processing*, volume 1898 of *Proc. SPIE*, pages 716–731, 1993.

[5] L. Lamport. *LaTeX: A Document Preparation System*. Addison-Wesley, Reading, Mass., 1994.

[6] N. Metropolis, A. W. Rosenbluth, M. N. Rosenbluth, A. H. Teller, and E. Teller. Equations of state calculations by fast computing machine. *J. Chem. Phys.*, 21:1087–1091, 1953.

[7] Frank Mittelbach, Michel Goossens, Johannes Braams, and David Carlisle. *The LaTeX Companion*. Addison-Wesley, Reading, Mass., 2004 (second edition).

[8] L. C. Perelman, J. Paradis, and E. Barrett. *Mayfield Handbook of Technical and Scientific Writing*. Mountain View, Mayfield, 1997. http://mit.imoat.net/handbook/.